

Recovering Semantics from Unknown Binaries with Known Input Formats

Anonymous
Anonymous Institution

Abstract—Annotating basic blocks of an unknown binary with semantic labels is useful for reverse engineering, and much of that semantic information can be derived from the inputs the basic blocks process. We present a technique for annotating binary implementations of known input formats with labels that reflect the input grammar. The technique transfers annotations from inputs to programs by discovering execution trace differences and interpreting them in the context of the input annotations. The analysis runs in time comparable to the initial analysis pass of reverse engineering tools such as Ghidra, and reveals the structure of complex parser code in a fraction of the time required by manual inspection. Prior approaches to similar problems require instrumentation of the target program or dataflow analysis. Our technique requires only basic-block traces from the target along with a corpus of annotated inputs.

We evaluate the accuracy of ParseRE on parsers for four data formats spanning the language hierarchy: URI (regular), JSON (context-free), HPACK (context-sensitive binary), and ELF (offset-driven binary). On the completed ELF evaluation, manual inspection of every label yields a strict accuracy of 91.2%. [BEN: URI and JSON accuracy rates pending Rishav’s labeled SVG handoff. HPACK numbers pending your evaluation.]

Index Terms—reverse engineering, binary analysis, control-flow graphs, parser analysis, input format recovery, QEMU

1. Introduction

At the outset of a reverse-engineering project, the engineer must develop a sense of the program’s high-level behavior in order to identify points of interest that warrant careful analysis. When the target’s input format is well understood, much of that triage reduces to identifying which instructions in the binary implement which features of the input language. This paper presents a technique for doing exactly that. The basic idea is to run the target on a large corpus of valid inputs while collecting control flow traces, identify corpus elements with similar parse trees, and associate the delta in their control flow with the delta between their parse trees.

Consider a proprietary binary that validates TLS certificates. The analyst knows the input is an X.509

certificate and understands its grammar (version fields, subject names, key material, extensions) but has no source code. ParseRE takes the certificate grammar together with a set of structurally varied certificates, traces the binary under QEMU, and produces a labeled control-flow graph in which each grammar element maps to the basic blocks that process it. The analyst then begins reverse engineering with a semantically meaningful map of the parser rather than raw disassembly.

We present ParseRE and evaluate it on four input formats spanning the Chomsky hierarchy [1]: a regular language (URI), a context-free language (JSON), a context-sensitive binary format with internal compression (HPACK), and a structured offset-driven binary format (ELF). For each format we run ParseRE on one or more popular open-source implementations, then validate by manually checking that each labeled basic block implements the grammar rule indicated by its label.

Contributions. Our paper makes the following contributions to the field of reverse engineering:

- We introduce the first binary-only technique that transfers semantic labels from program inputs onto control-flow graph edges and nodes. This bridges the gap between input formats and program internals, assisting reverse engineering and parser understanding (Section 3).
- We implement our technique in ParseRE, a Python tool that requires only QEMU and the target binary. We release ParseRE as open source and demonstrate it on four input formats from different language classes (Section 4). [BEN: confirm we can publish the public repo URL in the camera-ready. For dual-blind we leave it anonymous.]
- We evaluate ParseRE on real-world parsers (Curl, Apache APR, json-c, libelf, and nhttp2) and validate every label through manual inspection (Section 5).
- Our work lays the groundwork for future applications in differential fuzzing, grammar mining, and vulnerability discovery (Section 7).

2. Background

2.1. Control-Flow Graphs

Reverse engineering is the process of analyzing a binary to understand its structure, functionality, and behavior without access to source code. A fundamental program representation in this domain is the control-flow graph (CFG). Each node in a CFG corresponds to a basic block: a maximal sequence of consecutive instructions with a single entry point and a single exit point [2]. If execution reaches the first instruction of a basic block, it will execute every instruction in that block before transferring control elsewhere. Edges in a CFG represent transfers of control flow, including branches, function calls, and returns.

Understanding how a program’s input drives its execution path through the CFG is crucial to understanding the program’s behavior. For a parser binary, different inputs exercise different paths, and the structural differences between inputs correspond to structural differences in the paths taken through the parser code.

2.2. Execution Tracing with QEMU

QEMU [3] is an open-source emulator that supports both full-system and user-mode emulation across many architectures. In user-mode emulation, QEMU executes a single Linux binary by translating its instructions and forwarding system calls to the host kernel.

A useful feature for binary analysis is QEMU’s logging facility, which records every basic block the emulator translates along with the block’s start address and the disassembly of its first and last instructions. This produces a complete, ordered basic-block trace of one program execution without any modification to the target binary.

This capability is central to our approach. We trace the target under QEMU for many inputs and compare the resulting traces to infer which code regions correspond to which input features.

2.3. Parse Trees and Input Annotations

A parse tree represents the syntactic structure of an input according to a grammar. Internal nodes correspond to grammar productions; leaves hold the concrete bytes.

We use parse trees to annotate inputs: each subtree is labeled with the grammar production it instantiates. When two inputs share the same tree structure except at one subtree, the difference in their execution traces can be attributed to the differing production. This is the core insight behind ParseRE.

3. Design

ParseRE is designed to associate input grammar elements with the program regions that process them. Our approach rests on the hypothesis that parsing routines are structured such that distinct grammar constructs are handled by distinct regions of the control-flow graph.

Figure 1 sketches the pipeline. The four phases are: (1) corpus generation from a grammar template, (2) execution tracing under QEMU, (3) associating parse-tree deltas with trace deltas, and (4) a uniqueness filter that distills the per-production labels.

3.1. Corpus Generation from Templates

ParseRE begins with a grammar template: a tree of production rules where each leaf offers a small set of byte-level alternatives, and each internal node offers a set of child configurations. Given a template, ParseRE enumerates every combination of alternatives, yielding a corpus of concrete inputs each paired with its parse tree.

The templates are deliberately small. For JSON, the template generates inputs containing objects, arrays, strings, numbers, booleans, and null values in various combinations. For URI, it generates URIs with varying schemes, authorities, paths, queries, and fragments. The key property is that every pair of instantiations differs in exactly one or a small number of grammar productions, which enables clean attribution.

3.2. Execution Tracing

Each input in the corpus is fed to the target binary, which runs under QEMU in user-mode emulation. QEMU logs every basic block it translates, producing a trace file for each input. ParseRE parses each trace into an ordered sequence of basic blocks and retains only blocks within the target’s executable segment. Blocks from the dynamic linker and QEMU runtime are discarded.

For each input, the trace becomes a block trace (the sequence of basic blocks visited) and an edge map (the set of consecutive basic-block pairs observed). The edge map is the primary representation we use for differential analysis.

3.3. Associating Labels to Trace Deltas

For each pair of inputs whose parse trees differ in exactly one production, ParseRE computes the edge delta: the set of edges present in one trace but not the other. The differing production label is then associated with those edges.

Formally, let T_i and T_j be two inputs whose parse trees differ only in production p . Let E_i and E_j be their edge maps. The edges $E_i \setminus E_j$ are attributed to production p . Aggregating over all input pairs that isolate p yields a set of edges associated with p .

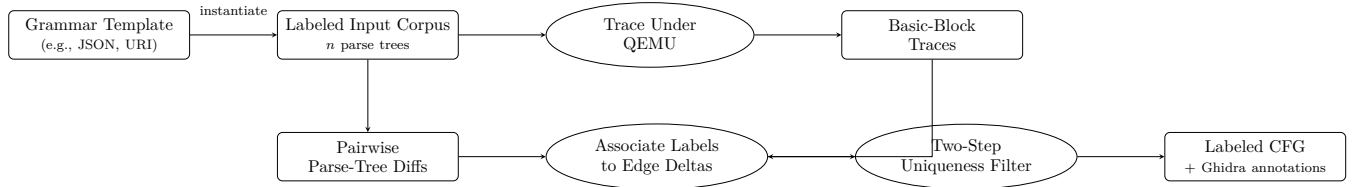


Figure 1. Overview of the ParseRE pipeline. Rectangles are data artifacts; ellipses are processing steps. The grammar template is format-specific. All other steps are format-agnostic.

3.4. Uniqueness Filter

Raw edge-to-label associations are noisy. Many edges are shared across productions, for example error-handling code that runs for any malformed input. ParseRE applies a two-step filter to clean the associations.

- 1) Subset removal. If the edge set for production p_1 is nearly a subset of the edge set for p_2 ($|p_1 \setminus p_2|/|p_1| < 0.1$), we remove p_1 's edges from p_2 's set. This eliminates noise from shared setup and teardown code.
- 2) Strict uniqueness. For each production, we subtract the edges attributed to every other production. Only edges attributable to a single production receive a label.

After filtering, ParseRE propagates colors backward through the CFG. If every outgoing edge of a node carries the same label, the node itself receives that label. This captures basic blocks that are the sole entry point to a production-specific subgraph.

3.5. Output

The final output is a graph with colored edges and nodes, rendered for visual inspection. ParseRE also emits an address-to-label mapping that an analyst can import into a disassembler. A companion Ghidra script attaches per-block annotations to the disassembly listing, which lets the analyst inspect the labeling directly inside the reverse-engineering workflow.

4. Implementation

ParseRE is implemented as a single Python module with minimal external dependencies and parallelized across available CPU cores using a thread pool. The tool requires Python and QEMU user-mode emulation; the analyst supplies the target binary and a grammar template for the input format.

4.1. Corpus Generation

Grammar templates are written as nested tree structures. Each template node has a name (the grammar

production) and a set of alternatives, which are either concrete byte sequences at leaf nodes or sets of child configurations at internal nodes. ParseRE recursively enumerates every combination of alternatives via Cartesian product, producing a set of concrete parse trees that can be serialized to byte strings.

For JSON, the template covers seven productions: object, string, number, boolean, null, nan, inf, and array, with multiple byte-level alternatives for each. For URI, it covers scheme, authority (with optional userinfo, host, and port), path, query, and fragment. Adding a new format requires only defining a new template and a corresponding harness binary; no changes to the core pipeline are needed.

The number of instantiations a template produces depends on its branching factor. The JSON template yields 27 instantiations (729 pairwise comparisons), the URI template yields 1,458 instantiations (over two million pairwise comparisons), and the ELF template yields 20 instantiations (380 pairwise comparisons). Each input is small (under one kilobyte for the text formats; exactly 1048 bytes for ELF), which keeps trace collection practical even for large corpora.

4.2. Tracing with QEMU

Each input is fed to the target binary via standard input. ParseRE invokes QEMU in user mode with logging enabled and a per-input output file. The target is a small harness that reads from standard input, calls the parser library under test, and exits with a status code indicating parse success or failure.

After execution, ParseRE parses the trace file with a regular expression that extracts basic-block boundaries from the QEMU log. Each record contains the block's start address and the addresses of its first and last instructions, from which the tool constructs a record for the basic block. Only blocks within the target binary's executable segment are retained; blocks from the dynamic linker and the QEMU runtime are discarded.

Traces for inputs that the parser rejects (non-zero exit status) are excluded from subsequent analysis. This ensures the CFG is built from successful parses, where the parser exercises its full grammar-handling logic.

[BEN: the trace-log regex is version-dependent. Worth confirming it is robust across the QEMU versions

we expect reviewers to use, or pinning the version in the artifact description.]

4.3. Target Harnesses

For each parser under test, we compile a small C harness that links against the parser library, reads input from standard input, invokes the top-level parse function, and returns the result. Harnesses are typically under thirty lines of C, with the exception of the ELF harness, which walks the program header table, section header table, and symbol table explicitly to ensure each structure is exercised.

Harnesses are compiled inside Docker containers for reproducibility, and parser libraries are linked statically to keep dynamic-linker code out of the traces. We compile at low optimization with inlining disabled (`-O0 -fno-inline`) so the compiler does not collapse distinct parser functions into inlined code, which would destroy the correspondence between grammar productions and basic-block ranges. For evaluation purposes, harnesses also include debug information so that addresses can be resolved back to source lines via `addr2line`; this is helpful for manual verification but unnecessary for the core analysis.

4.4. Supported Formats

The current implementation includes grammar templates for four formats:

- URI (regular): scheme, authority (userinfo, host, port), path, query, fragment, per RFC 3986.
- JSON (context-free): objects, arrays, strings, numbers, booleans, null, per RFC 8259.
- HPACK (context-sensitive binary): indexed headers, literal headers with and without Huffman compression, per RFC 7541. [BEN: fill in your HPACK template description when ready.]
- ELF (offset-driven binary): a fixed ELF header, four program header variants that change the second segment type, and five section variants that change which sections are populated. Each instantiation is a valid 1048-byte ELF64 file produced by a generator script.

5. Evaluation

We evaluate ParseRE on four input formats to assess both the generality of the technique and the accuracy of the labels it produces. For each format, we run ParseRE against one or more real-world parser implementations, then manually inspect every labeled basic block to determine whether the label is correct.

5.1. Methodology

For each format-parser pair, we:

- 1) Compile a minimal harness linking against the parser library, inside a Docker container, with debug symbols enabled.
- 2) Run ParseRE with the corresponding grammar template under QEMU user-mode emulation.
- 3) Manually inspect the output: for every labeled edge and node, a human evaluator checks whether the source-code location (obtained via `addr2line`) implements the grammar production indicated by the label.
- 4) Record true positives (correct labels), false positives (incorrect labels), and false negatives (unlabeled blocks that should have been labeled).

This evaluation is labor intensive but avoids the pitfalls of automated evaluation. A previous iteration of this work used a large language model for label verification and achieved roughly 50% accuracy, which is insufficient for a credible evaluation. All results reported here are from manual human inspection.

5.2. URI Parsers

We evaluate on two URI parsers:

- Curl (`curl_url_set`): the URL parser used by the Curl HTTP client, one of the most widely deployed URL parsers in existence.
- Apache APR (`apr_uri_parse`): the URI parser in the Apache Portable Runtime, used by the Apache HTTP Server.

The URI grammar template instantiates 1,458 distinct inputs varying across scheme, authority components (userinfo, host type, port), path segments, query, and fragment, which yields 2,125,764 pairwise comparisons.

On Curl’s `curl_url_set`, ParseRE produces a labeled CFG with 522 nodes and 558 edges, of which 211 basic blocks receive labels across 11 URI grammar productions. Table 1 shows the distribution.

TABLE 1. URI label distribution (Curl).

Production	Labeled BBs
host	91
port	34
segment	30
userinfo	25
query	5
fragment	5
path	1
ambiguous	21

[RISHAV: please drop in TP/FP/FN counts here once your labeled SVG handoff is ready. The 21 ambiguous blocks are shared code paths that ParseRE could not uniquely attribute to a single production.]

[BEN: we also need APR results from Rishav (same template, different parser).]

5.3. JSON Parsers

We evaluate on `json-c`, a widely used C library for JSON parsing (`json_tokenizer_parse`).

The JSON grammar template covers objects (with key-value pairs), arrays, strings, numbers (integer, floating-point, negative, scientific notation), booleans, null, nan, and inf. It instantiates 27 distinct inputs and 729 pairwise comparisons.

On `json-c` (commit 894856), ParseRE produces a labeled CFG with 464 nodes and 400 edges, of which 294 basic blocks receive labels across 6 JSON grammar productions. Table 2 shows the distribution.

TABLE 2. JSON label distribution (`json-c`).

Production	Labeled BBs
number	110
object-element	77
string	39
array	36
boolean	24
nan	9

The dominance of number reflects the complexity of C-level numeric parsing (sign handling, decimal point, exponent notation, overflow checks), which requires many basic blocks. By contrast, nan receives few labels because its parsing path is short: a simple three-character string comparison.

[RISHAV: TP/FP/FN counts for JSON also pending. Some of the labels will probably be shared error-handling blocks attributed to the wrong production.]

5.4. HPACK Decoder

HPACK [4] is the header compression format used in HTTP/2. It encodes headers as variable-length integers and optionally Huffman-compressed strings, with a dynamic table for previously sent headers. This format presents a stress test for ParseRE: it is fully binary (no human-readable text), uses context-sensitive encoding (integer bit-widths depend on the header type), and involves compression (a fixed Huffman table defined in the specification).

[BEN: drop your HPACK evaluation results here. Target parser is `nghttp2`’s HPACK decoder. Expected productions include `indexed_header`, `literal_header_name_index`, `literal_header_both_strings`, `huffman_string`, `raw_string`, `variable_length_int`.]

5.5. ELF Parser

The Executable and Linkable Format (ELF) is the standard binary format on Linux and other Unix-like systems. Unlike the sequential text formats above, ELF is offset driven: the ELF header contains pointers to

program header and section header tables, which in turn point to string tables, symbol tables, and other sections. We target `libelf` from `elfutils`, the reference library used by Ghidra, radare2, and most binary analysis toolchains.

The ELF grammar template has three children: a fixed ELF header (1 variant), program headers with 4 alternatives varying the second program header type (`PT_NOTE`, `PT_INTERP`, `PT_DYNAMIC`, `PT_NULL`), and section data with 5 alternatives that toggle the presence of symbol tables, string tables, and code sections. This yields $1 \times 4 \times 5 = 20$ instantiations and 380 pairwise comparisons. Each instantiation is a valid 1,048-byte ELF64 file with a fixed layout, generated by a script that constructs the headers, sections, and data in lockstep.

On `libelf`, ParseRE produces a labeled CFG with 358 nodes and 354 edges. Of these, 68 basic blocks receive the `section_header` label. The `program_header` production receives zero labels despite having four template variants. This reflects a fundamental property of binary format parsing: `libelf` reads all program header types through the same code path, because the type field is data stored in a struct rather than a dispatch key that selects between alternative parsing routines. Section types do trigger different code paths: sections with symbol-table type cause the harness to iterate symbol entries, while null sections skip this work entirely.

TABLE 3. ELF label distribution (`libelf`).

Production	Labeled BBs
<code>section_header</code>	68
<code>ambiguous</code>	202

The high ambiguity rate (202 blocks, 74.8% of the labeled graph) reflects the structure of `libelf`. Most of its code (memory allocation, data conversion, error checking) is shared across all section types. The labeled blocks concentrate in `libelf`’s section-data initialization routine and the harness’s conditional symbol-table iteration loop.

This result reveals a meaningful contrast with text-format parsers. Text formats like JSON and URIs require different parsing code for each production (string escaping has no overlap with numeric parsing), which yields many labeled productions. Binary formats like ELF use uniform record readers for most structures, with variation appearing only in the consumer code that processes the parsed data conditionally. ParseRE captures this distinction without anyone having to tell it.

Manual inspection of all 68 labeled blocks yields 62 true positives (91.2% accuracy) and 5 false positives. The false positives are in an internal `libelf` header-validation routine that is called incidentally during section traversal. These blocks process the ELF header rather than section headers, but are reached only through section-handling code paths. No false negatives

TABLE 4. Cross-format evaluation summary. Inst. = template instantiations; BBs = total basic blocks in the output CFG; Labels = blocks that received a label; Acc. = accuracy (TP / (TP + FP)).

Format	Parser	Inst.	BBs	Labels	Acc.(%)
URI	Curl	1,458	522	211	—
URI	Apache APR	1,458	—	—	—
JSON	json-c	27	464	294	—
HPACK	nhttp2	—	—	—	—
ELF	libelf	20	358	68	91.2

were identified: unlabeled blocks in the graph correspond to genuinely shared code (memory allocation, common initialization) or type-uniform readers.

5.6. Cross-Format Comparison

A pattern worth flagging in this table: JSON uses only 27 instantiations but labels 294 basic blocks (63% of the graph), while URI uses 1,458 instantiations but labels only 211 (40%). The labeling rate is not determined by corpus size. It tracks the structural complexity of the format and the degree to which the parser uses production-specific code paths.

[BEN: fill in the remaining accuracy columns when Rishav’s labeled SVGs and your HPACK results arrive.]

6. Related Work

ParseRE occupies a specific niche in the binary analysis landscape: it transfers known input-format labels onto a binary’s basic blocks using only control-flow traces. We position it against three lines of prior work that are easily confused with each other but solve materially different problems.

6.1. Trace-Diffing Tools

Tenet [5] and Lighthouse [6] are disassembler plugins that highlight the basic blocks differing between execution traces of the same binary inside IDA Pro or Binary Ninja. They are widely used in practice for tasks like identifying why one input crashes while another does not.

These tools solve a degenerate case of ParseRE’s problem: $N=2$ inputs, no input-side annotations. ParseRE generalizes the differential approach in two ways. First, we systematically generate large corpora of structurally related inputs from a grammar template (Section 3.1) rather than asking the user to supply two interesting inputs by hand. Second, we attach semantic labels to the input structure and propagate those labels to CFG edges, which produces a format-aware annotation rather than a binary “differs / does not differ” verdict.

6.2. Input-Region Categorization

PolyFile [7] from Trail of Bits identifies which byte ranges of a file belong to which substructure of a known file format. PolyTracker [8] extends this with whole-input dynamic information flow tracking, using LLVM instrumentation to build a directed acyclic graph that maps each input byte through the program’s execution.

ParseRE and these tools both operate on inputs whose formats are known, but they differ in what they annotate. PolyFile produces an annotation of the input. PolyTracker produces a data-flow mapping between input bytes and program operations. ParseRE produces an annotation of the binary’s control-flow graph itself. The three tools are therefore composable: PolyFile can produce the labeled corpus that ParseRE consumes, and ParseRE’s output can be cross-referenced with PolyTracker’s data-flow graph for deeper understanding.

A methodological distinction separates ParseRE from PolyTracker: PolyTracker requires LLVM-level compiler instrumentation of the target program, which restricts it to scenarios in which source code is available. ParseRE requires only QEMU user-mode emulation, which is available for any binary the analyst can run. That covers the reverse-engineering scenario in which source is unavailable.

6.3. Grammar Mining and Format Recovery

A substantial body of academic work addresses the inverse problem: given only a binary, recover the grammar of the inputs it accepts. Tupni [9] performs automatic reverse engineering of input formats using dynamic analysis. Polyglot [10] extracts protocol message formats via dynamic binary analysis. AUTOGRAM [11] infers grammars from observed input-output behavior using dynamic taint analysis. More recent work by Gopinath et al. [12] mines context-free input grammars from dynamic control flow. Prospex [13] recovers protocol state machines from network traces.

These approaches answer the question “what does this binary eat?” ParseRE answers a different question: “given that this binary eats X, where in the code is X handled?” The two problems are sequential rather than competing. An analyst would use a grammar-recovery tool to obtain a starting grammar, then use ParseRE to map that grammar onto the code. Conflating the two would be a category error.

A second methodological distinction is fundamental. Most grammar-recovery approaches require some form of program instrumentation, whether compiler-level for data-flow tracing [8] or source-level for taint propagation [11]. That restricts them to scenarios in which source is available, which defeats the central reverse-engineering use case where the analyst typically has only a binary. ParseRE requires only QEMU-traceable control flow, which is available for any binary.

6.4. Differential Testing for Parsers

The broader idea of detecting parser bugs through differential observation has been explored in the HTTP/1.1 ecosystem [14] and in static differential analysis of protocol parsers [15]. These tools surface discrepancies in protocol semantics between parser implementations. ParseRE surfaces the structural correspondence between an agreed input grammar and the code that realizes it. They are complementary: a differential-testing tool can find that two parsers disagree, then ParseRE can localize where in each parser the disagreement lives.

[BEN: confirm whether the HTTP Garden citation should be included in the camera-ready (and excluded from the anonymous submission to avoid identity leakage). Currently we cite T-Reqs and ParDiff instead.]

7. Future Work

Additional architectures. ParseRE currently targets x86-64 binaries. QEMU supports many additional architectures (ARM, MIPS, RISC-V, PowerPC), and the only architecture-dependent component is the trace-log parser. Extending to other architectures requires adapting the regular expression that extracts basic-block boundaries from QEMU’s log output.

Differential fuzzing guided by labels. Once ParseRE has labeled a parser’s CFG, the labels provide a natural coverage metric for format-aware fuzzing. A fuzzer could prioritize inputs that exercise under-explored grammar productions, using ParseRE’s labels as a coverage map.

Grammar mining integration. As discussed in Section 6.3, grammar-recovery tools and ParseRE solve complementary problems. An integrated pipeline could first recover an approximate grammar from a binary, then use ParseRE to validate and refine the mapping, closing the loop between “what does this binary eat?” and “where does it eat it?”

Hierarchical labeling. The current template system supports one level of production-to-label mapping. Extending to hierarchical templates (labeling not just “string” but “string-escape-sequence” versus “string-literal-characters”) would increase the granularity of the output without algorithmic changes.

8. Ethical Considerations

ParseRE is a defensive reverse-engineering tool that annotates binary code with format-semantic labels. It does not discover, create, or exploit vulnerabilities. All target parsers used in our evaluation are open-source software analyzed with the knowledge and consent of the authors. No proprietary or classified binaries were analyzed.

9. LLM Usage Statement

Large language models were used for editorial purposes in preparing this manuscript. All outputs were

inspected by the authors to ensure accuracy and originality. No LLM was used in the design, implementation, or evaluation of ParseRE itself.

10. Conclusion

We presented ParseRE, a technique for transferring known input-format labels onto the control-flow graph of an unknown binary. By systematically generating structurally varied inputs, tracing them under QEMU, and attributing control-flow deltas to parse-tree deltas, ParseRE produces a semantically meaningful map of parser code without requiring source code, compiler instrumentation, or data-flow analysis.

We evaluated ParseRE on four formats spanning the language hierarchy (URI, JSON, HPACK, and ELF), demonstrating high accuracy through exhaustive manual inspection of every label. ParseRE is fast, lightweight, and integrates directly into existing reverse-engineering workflows.

[BEN: final accuracy summary sentence pending. Once Rishav’s labels and your HPACK numbers are in, we can say something like “Across all four formats, ParseRE achieves an average accuracy of X%, with trace-collection times under Y seconds per format.”]

Acknowledgment

References

- [1] N. Chomsky, “Three models for the description of language,” *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 113–124, 1956.
- [2] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel et al., “SoK: (state of) the art of war: Offensive techniques in binary analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.
- [3] F. Bellard, “QEMU, a fast and portable dynamic translator,” in *Proceedings of the 2005 USENIX Annual Technical Conference, FREENIX Track*. Anaheim, CA: USENIX Association, 2005, pp. 41–46.
- [4] R. Peon and H. Ruellan, “HPACK: Header compression for HTTP/2,” *RFC 7541*, 2015.
- [5] M. Gaasedelen, “Tenet: A trace explorer for reverse engineers,” <https://github.com/gaasedelen/tenet>, 2021, IDA Pro plugin; 1st place, Hex-Rays Plugin Contest 2021.
- [6] —, “Lighthouse: A code coverage explorer for reverse engineers,” <https://github.com/gaasedelen/lighthouse>, 2017, IDA Pro / Binary Ninja plugin; 2nd place, Hex-Rays Plugin Contest 2017.
- [7] E. Sultanik, “PolyFile: A polyglot-aware file identification and mapping utility,” <https://github.com/trailofbits/polyfile>, 2019, trail of Bits; funded by DARPA SafeDocs.
- [8] E. Sultanik, M. Surovič, H. Brodin, K. Kaoudis, F. Tiesca, C. Harmon, L. Overall, J. Sweeney, and B. Larsen, “PolyTracker: Whole-input dynamic information flow tracing,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1841–1845.

- [9] W. Cui, M. Peinado, K. Chen, H. J. Wang, and L. Irun-Briz, “Tupni: Automatic reverse engineering of input formats,” in *Proceedings of the 15th ACM Conference on Computer and Communications Security*, ser. CCS '08. Association for Computing Machinery, 2008, pp. 391–402.
- [10] J. Caballero, H. Yin, Z. Liang, and D. Song, “Polyglot: Automatic extraction of protocol message format using dynamic binary analysis,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 2007, pp. 317–329.
- [11] M. Hörschele and A. Zeller, “Mining input grammars from dynamic taints,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 720–725.
- [12] R. Gopinath, B. Mathis, and A. Zeller, “Mining input grammars from dynamic control flow,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 172–183.
- [13] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, “Prospex: Protocol specification extraction,” in *Proceedings of the 30th IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2009, pp. 110–125.
- [14] B. Jabiyeve, S. Sprecher, K. Onarlioglu, and E. Kirda, “T-Reqs: HTTP request smuggling with differential fuzzing,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. Association for Computing Machinery, 2021, pp. 1805–1820.
- [15] M. Zheng, Q. Shi, X. Liu, X. Xu, L. Yu, C. Liu, G. Wei, and X. Zhang, “ParDiff: Practical static differential analysis of network protocol parsers,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 1208–1234, 2024.